



SESSION 20 · PHASE 3 — ADVANCED & ENTERPRISE

Security — Part 2

Client Credentials Flow · Spring Security OAuth2 Resource Server · Retiring JwtAuthFilter



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Monday

TIME

3:00 – 5:30 PM

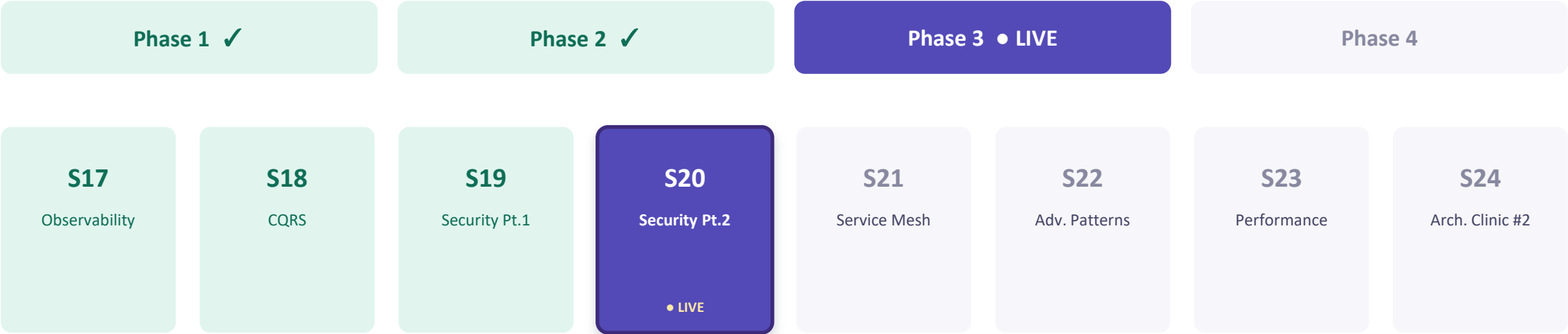
DURATION

2.5 Hours

MODE

Online (Live)

Where We Are



Today's session builds directly on S19's Keycloak setup — full roadmap lives in the Kickoff deck only.

Closing the Loop Opened in Sessions 3 and 19

Session 3 built `JwtAuthFilter` as a deliberate placeholder — hand-rolled signature validation against a hardcoded secret.

Session 19 gave us a real Identity Provider and human login via the Authorization Code Flow.

Today we close both loops: the Gateway stops hand-validating JWTs and becomes a real Spring Security Resource Server that trusts Keycloak's published keys — and we add the missing half of OAuth2: the flow with no human in it at all.

Before S20 vs After S20

BEFORE S20

Custom JwtAuthFilter (Session 3)

- ❗ Hardcoded shared secret in application.yml
- ❗ No real user accounts — only test tokens
- ❗ No token revocation before natural expiry
- ❗ No login UI, no password reset flow

🗑️ DELETED TODAY

AFTER S20

Keycloak OAuth2 Resource Server

- ✓ RSA public keys via JWKS — no shared secret
- ✓ Real users + service accounts from Keycloak
- ✓ Standards-based token issuance & validation
- ✓ Client Credentials for service-to-service calls

📅 ADDED TODAY

Three Questions Session 3's Security Could Not Answer



Where do real customers sign up?

No signup form — only pre-signed test tokens generated with a Java snippet.



What if a customer forgets their password?

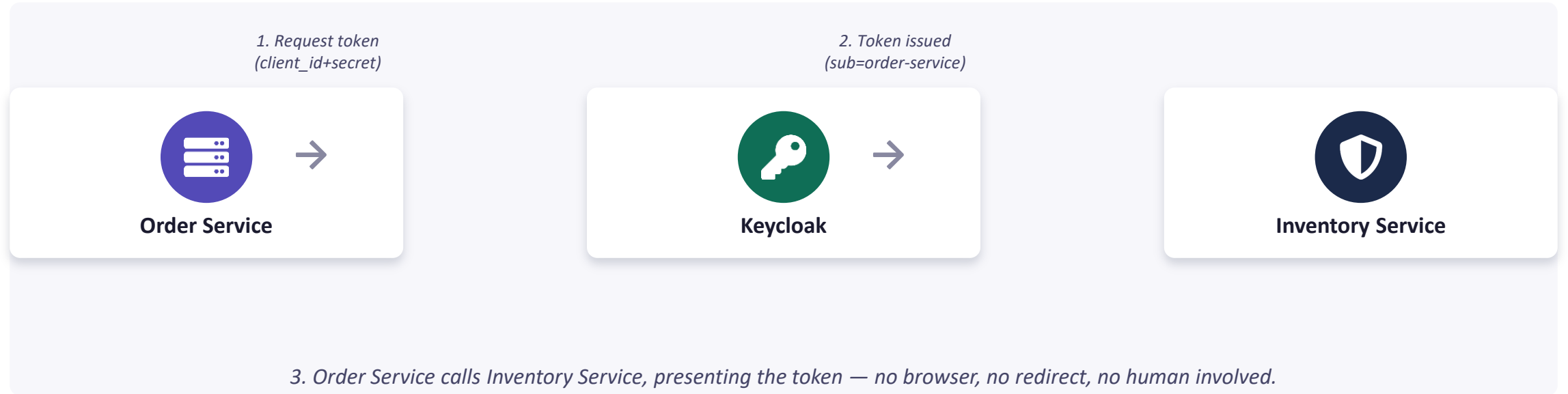
No reset flow — because there was never a real user store behind the filter.



Can we revoke ONE stolen token early?

No — JwtAuthFilter only checks a signature and an expiry timestamp.

Client Credentials Flow — 3 Steps, No User



Client Credentials vs Authorization Code (Session 19)

Aspect	Authorization Code (S19)	Client Credentials (Today)
Who authenticates?	A human user (Resource Owner)	The Client itself — no user
Browser / redirect?	Yes — login page, then redirect back	No — a single server-to-server POST
Token represents	The user (sub = user ID, roles)	The Client (sub = client ID)
Platform example	Customer logs into the storefront	Order Service calling Inventory Service

Where Should JWT Validation Happen?

Validate at Gateway only?



Simpler — single point of enforcement

Validate at each service?



Defense in depth — no single point of failure

Our platform's choice?



Gateway validates first + Resource Server on sensitive services

Stateless JWT vs Opaque Token

Need to revoke tokens immediately?



Opaque token (introspection call each request)

Performance, no network call per request?



JWT (stateless — verify locally via public key)

Our choice for this platform?



JWT (stateless) — simpler for training, aligns with S3's original design intent

order-service: Add Resource Server Dependency

```
<!-- services/order-service/pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>

<!-- spring-boot-starter-security is pulled in transitively -->
<!-- No JWT dependency needed here — Spring Security handles JWKS -->
```

Same starter used on the Gateway (slide 15) — one dependency, two services, consistent config.

order-service: Configure the Issuer URI

```
# application.yml - order-service
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://keycloak:8090/realms/microservices-pro
          # Spring Security auto-discovers the JWKS endpoint from this URI

# DELETE - no longer used anywhere:
# jwt:
#   secret: microservices-pro-course-secret-key-2024...
```

order-service: SecurityConfig — Role Extraction

```
// SecurityConfig.java — order-service
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()))
            .build();
    }
}
```

realm_access.roles is mapped automatically by Spring Security's default converter once 'roles' scope is present.

order-service Now Trusts Keycloak, Not a Hardcoded Secret

NO TOKEN

```
$ curl http://order-service:8082/api/orders  
  
HTTP/1.1 401 Unauthorized  
{"error":"unauthorized"}
```

VALID KEYCLOAK TOKEN

```
$ curl -H "Authorization: Bearer $TOKEN" \  
      http://order-service:8082/api/orders  
  
HTTP/1.1 200 OK  
[{"orderId":"ORD-9001", ...}]
```

Why Validate at the Gateway Too?

Recall the Trust Boundary principle from Session 3: the Gateway is the ONLY service the outside world knows about. Every request enters through it first.

What changes today: the trust no longer rests on a hardcoded secret shared across files — it rests on Keycloak's published public keys (JWKS). Downstream services can independently verify the exact same token, with no coordination and no shared file.

Same pattern. New foundation.

api-gateway: Replace JwtAuthFilter

```
<!-- api-gateway/pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>

<!-- DELETE these two Gateway classes entirely — not commented out: -->
<!--   filters/JwtAuthFilter.java      (Session 3) -->
<!--   util/JwtUtil.java                (Session 3) -->
```

Deleting, not commenting out — half-retired security code is a liability, not a safety net.

api-gateway: Resource Server Config + Protected Routes

```
# application.yml — api-gateway
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://keycloak:8090/realms/microservices-pro

# Public routes (e.g. GET /api/products) remain permitAll,
# same pattern established in Session 3's PUBLIC_ROUTES list
```

Two Independent Layers Reject the Same Bad Request

Request path	No token
Direct to order-service (bypassing Gateway)	401 — Resource Server rejects it
Through api-gateway → order-service	401 — Gateway rejects it BEFORE it ever reaches order-service

Two independent security layers — neither depends on the other being correctly configured.

Refresh Token Rotation — Not Implemented Manually



Refresh Token rotation is handled by Keycloak automatically.

This course does not implement manual refresh-token handling in application code — Keycloak's token endpoint and session management already do this correctly. Writing your own rotation logic on top would duplicate, and likely weaken, what the Identity Provider already guarantees.

Wire Resource Server + Retire JwtAuthFilter

1

Wire the Gateway as a Resource Server

Task 1

12 min

2

Preserve X-User-Id / X-User-Role header enrichment

Task 2

5 min

3

Client Credentials for Order Service → Inventory Service

Task 3

10 min

4

Role-based route + unit tests

Task 4

3 min + homework

Acceptance Criteria

- ✓ JwtAuthFilter.java and JwtUtil.java no longer exist in the repository
- ✓ GET /api/products (no token) still returns 200 — public route preserved
- ✓ A valid Keycloak-issued customer token successfully authenticates a protected route
- ✓ X-User-Id and X-User-Role headers still arrive at Order Service, sourced from the real JWT
- ✓ Order Service obtains a Client Credentials token and calls Inventory Service with no user in context
- ✓ A customer token receives 403 on an ADMIN-only route; an admin token receives 200

What We Added Today

 CAPABILITY ADDED

Federated Identity Everywhere

Human login (S19) and service-to-service auth (today) both trace back to one Identity Provider — no hardcoded secrets remain anywhere in the Gateway.

 SERVICES UPDATED

api-gateway · order-service

Gateway: JwtAuthFilter retired, Resource Server wired. Order Service: Resource Server + Client Credentials to Inventory Service.



Config+Eureka



Gateway



Resilience



Feign+Saga



K8s Pipeline



Observability



CQRS



NEW

Security

Common Issues & Solutions

Issue	Solution
Gateway fails to start: "Unable to resolve issuer"	Keycloak must be running and reachable BEFORE the Gateway starts — JWKS is fetched at startup.
Valid token rejected with 401 after cutover	issuer-uri must match the token's iss claim EXACTLY (protocol, host, port, realm).
realm_access.roles missing — role route always 403	Client Scopes must include 'roles', and the user needs the role explicitly assigned.
Client Credentials token obtained but call still rejected	Inventory Service must trust the same issuer-uri, or trust propagated headers instead.
OrderServiceTokenClient requests a new token every call	Check the expiry-caching logic — Instant.now() comparison must actually persist across calls.

Homework & Checkpoint Commit

🔗 DEFINITION OF DONE — SESSION 20

- 🔗 JwtAuthFilter.java / JwtUtil.java deleted
- 🔗 oauth2-resource-server configured (issuer-uri)
- 🔗 Public route still works without a token
- 🔗 X-User-Id/Role headers preserved
- 🔗 Client Credentials: Order → Inventory verified
- 🔗 ADMIN route: 403 customer / 200 admin

🔗 PRE-SESSION 21 READING

Topic: Service Mesh Fundamentals + Istio

Knowledge Check questions for Session 21:

1. *What is a sidecar proxy, and why does Istio inject one?*
2. *What is mTLS, and how does it differ from standard HTTPS TLS?*
3. *What would mesh-level resilience mean vs Resilience4j (S4)?*

Commit: session-20: retire-jwtauthfilter-add-oauth2-resource-server-and-client-credentials



Session 20 Complete

Production Security Fully Wired — Human Login + Service-to-Service Auth

mTLS via Istio adds another security layer on top of what we built in S19–S20.

Next: Session 21 — Service Mesh: Istio · Wednesday, 3:00–5:30 PM, Online